CS 432: Databases

FixIIT – Maintenance Portal

Assignment 2 - Module B

Submitted by: Team Codebase

Abhitej Singh Bhullar (23110009)
Shiv Patel (23110302)
Shivansh Shukla (23110303)
Soham Shrivastava (23110315)
Arin Mehta (23110038)

Github Link:
https://github.com/ArinMehta/FixIIT

Video Link: https://drive.google.com/file/d/1QM8AQvYB7yuoPPMKjK_0YnkT7UAqC0oM/view?usp=drive_link

Contents

1 Project Objective **3**

2 System Overview **3**
2.1 Functional Scope . . . 3
2.2 Implementation Components . . . 4

3 Methodology and Approach **4**
3.1 Reuse of Assignment 1 Schema . . . 4
3.2 New Additions for Module B . . . 5
3.3 Why the Implementation Stayed Minimal . . . 5

4 Authentication and Session Management **5**
4.1 Login Flow . . . 5
4.2 Session Validation . . . 6

5 Role-Based Access Control **6**
5.1 Role Model . . . 6
5.2 Backend Enforcement . . . 7
5.3 Validation of RBAC . . . 7

6 Member Profile Feature **7**
6.1 Design Rationale . . . 7
6.2 Schema Support . . . 8
6.3 API Support . . . 8
6.4 UI Integration . . . 8

7 Audit Logging **9**
7.1 Need for Persistent Auditability . . . 9
7.2 Audit Table Design . . . 9
7.3 Write Coverage . . . 9

8 Tamper Detection via Triggers **10**
8.1 Why Application Logs Alone Are Not Enough . . . 10
8.2 Trigger Design . . . 10
8.3 Before/After Payloads . . . 10
8.4 Validation of Tamper Detection . . . 11

9 Query Optimization and Indexing **11**
9.1 Optimization Philosophy . . . 11
9.2 High-Value Query Patterns . . . 11
9.3 Final Index Set . . . 11
9.4 Why These Indexes Were Chosen . . . 12

9.5 Redundant Index Removal 12

10 Benchmarking Methodology **12**
10.1 Need for Reproducible Evidence 12
10.2 Pipeline Steps 12
10.3 Generated Artifacts 13

11 Benchmark Results and Interpretation **13**
11.1 Interpretation 13

12 EXPLAIN Query-Plan Analysis **14**
12.1 Why EXPLAIN Was Used 14
12.2 Queries Analyzed 14
12.3 Interpretation of Plans 14

13 Testing and Validation **14**
13.1 Authentication and Route Protection Tests 14
13.2 Audit and Tamper Tests 15
13.3 Benchmark Validation 15

14 Tools and External Sources Used **15**
14.1 Tools Used 15
14.2 External Assistance 16

15 Repository README and Submission Notes **16**

16 Conclusion **16**

17 Team Contributions **17**

1 Project Objective

The objective of Assignment 2 Module B is to convert the FixIIT database design from Assignment 1 into a working full-stack database application with security, auditability, and performance-oriented improvements. The system is intended for campus maintenance management, where members can raise complaints, administrators can monitor and control operations, and the system itself can provide evidence of correctness, traceability, and optimization.

In Assignment 1, the focus was on schema design, normalization, constraints, and conceptual modelling. In Module B, the focus shifts from schema-only design to a functioning application layer built over the same schema. Therefore, the goal of this phase is not to redesign the domain, but to bring the existing schema to life with web interfaces, authenticated APIs, role-based access control, audit logging, direct-database tamper detection, and evidence-based query optimization.

The implementation was guided by the following four goals:

- building a working web interface and API layer over the FixIIT schema,
- enforcing authentication and role-based access control locally,
- ensuring that every important modification is traceable and that unauthorized direct database changes can be identified,
- optimizing query execution using actual workload-driven indexes, measured benchmarks, and EXPLAIN-based plan analysis.

2 System Overview

FixIIT is a campus maintenance portal where members report infrastructure or utility issues such as electrical faults, plumbing issues, or other maintenance complaints. The final Module B implementation uses **Flask** as the backend application layer, **MySQL** as the persistent storage engine, and server-rendered HTML templates for the user-facing pages.

The Module A schema already contained the main operational entities of the maintenance domain: members, roles, tickets, assignments, comments, feedback, categories, statuses, and locations. Module B extends this by adding the necessary application support components required for a secure and verifiable system.

2.1 Functional Scope

The implemented application supports the following operational scope:

- user login with locally validated session tokens,
- ticket creation and user-specific ticket listing,

- administrator access to global ticket management,
- a member profile page satisfying the "Member Portfolio" requirement,
- persistent database-backed logging of modifying operations,
- identification of unauthorized direct SQL modifications,
- performance benchmarking before and after indexing,
- EXPLAIN plan capture for major application queries.

2.2 Implementation Components

The final implementation is organized into the following layers:

- **Backend routes**: implemented in `Module_B/app/api.py`
- **Authentication**: implemented in `Module_B/app/auth.py`
- **RBAC decorators**: implemented in `Module_B/app/rbac.py`
- **Database helpers**: implemented in `Module_B/app/database.py`
- **Templates**: dashboard, admin panel, and profile pages under `Module_B/templates/`
- **SQL setup and index scripts**: under `Module_B/sql/`
- **Benchmark and EXPLAIN artifacts**: under `Module_B/performance/results/`

3 Methodology and Approach

The implementation followed an incremental, schema-preserving approach rather than a full redesign. This decision was deliberate because Assignment 1 had already produced a strong normalized schema for the maintenance domain. Therefore, Module B was designed as an application-layer realization of the existing system, with only minimal additional tables and scripts required to satisfy the new requirements.

3.1 Reuse of Assignment 1 Schema

The Assignment 1 schema already captured the maintenance workflow correctly, including:

- member registration and identity,
- issue categories,
- ticket lifecycle through statuses,
- locations of complaints,

- assignment of technicians,
- comments and feedback,
- role associations through `member_roles`.

Instead of modifying the domain aggressively, Module B extends it in a controlled way so that the underlying conceptual model remains consistent.

3.2 New Additions for Module B

To support the new assignment requirements, the following additions were made:

- `Credentials`: stores username and password hash for local authentication,
- `member_portfolio`: stores user profile information for the profile page,
- `db_change_audit`: stores persistent write-audit records,
- MySQL triggers on mutable tables to classify changes as `API` or `DIRECT_DB`,
- index-creation and index-removal scripts used by the benchmark workflow.

3.3 Why the Implementation Stayed Minimal

A key design goal was to avoid overbuilding the system. The assignment emphasizes methodology, functionality, and approach rather than unnecessary complexity. Therefore:

- the existing ticketing system was preserved,
- the profile requirement was implemented as a domain-consistent "My Profile" page,
- audit and tamper-detection were combined into one coherent database-backed mechanism,
- optimization focused only on high-value queries actually used by the app.

This approach kept the implementation aligned with the assignment while maintaining consistency with the FixIIT domain.

4 Authentication and Session Management

4.1 Login Flow

The application uses a local login flow backed by the `Credentials` table. When a user submits a username and password to `POST /login`, the backend performs the following steps:

1. fetch the member authentication record by joining `Credentials`, `members`, `member_roles`, and `roles`,

2. hash the submitted password using the same SHA-256 scheme used by the stored credential row,
3. compare the resulting hash with the stored `password_hash`,
4. if valid, issue a JWT session token containing member identity and role metadata.

A successful login returns:

- `member_id`,
- `username`,
- `name`,
- `is_admin`,
- `role_codes`,
- `session_token`.

This token is then used in the `Authorization: Bearer <token>` header for protected routes.

4.2 Session Validation

The assignment required local session validation on every API request. This was implemented using route decorators. Every protected route validates the authorization header and token before executing its underlying business logic. This ensures that requests are not trusted based on frontend behavior alone.

The final implementation was verified using route tests showing:

- unauthenticated access to `/dashboard` returns 401,
- unauthenticated access to `/admin` returns 401,
- valid authenticated access to `/dashboard` succeeds,
- admin access to `/admin` succeeds only with an admin token.

5 Role-Based Access Control

5.1 Role Model

The system enforces two meaningful application roles for Module B evaluation:

- **Admin**
- **Regular User**

Role information is derived from the `member_roles` and `roles` tables and embedded in the login token payload.

5.2 Backend Enforcement

A major concern in systems like this is mistaking frontend hiding for actual authorization. To avoid this, role enforcement is performed on the backend using decorators:

- `@login_required` for authenticated user routes,
- `@admin_required` for admin-only routes.

The final protected routes include:

- `/dashboard` - login required,
- `/admin` - admin required,
- `/admin/tamper-events` - admin required,
- ticket update and delete routes - admin required,
- profile update route - login required.

5.3 Validation of RBAC

The RBAC mechanism was validated using direct route tests:

- `/admin` without a token returned 401 Unauthorized,
- `/admin` with a regular-user token returned 403 Forbidden,
- `/admin` with an admin token returned 200 OK,
- `/dashboard` without a token returned 401 Unauthorized,
- `/dashboard` with a valid token returned 200 OK.

These tests confirm that access control is enforced by the backend and not merely suggested by the frontend UI.

6 Member Profile Feature

Although the FixIIT domain is a maintenance portal rather than a professional networking system, Module B explicitly required a web UI supporting a Member Portfolio. To satisfy the requirement without distorting the domain, this was implemented as a **My Profile** feature.

6.1 Design Rationale

The interpretation taken was that "Member Portfolio" in the context of FixIIT can be represented as a member-specific profile page. This keeps the implementation assignment-compliant while preserving application consistency.

6.2 Schema Support

A new table, `member_portfolio`, was added with a one-to-one relation to `members`. It stores:

- name,
- email,
- contact no,
- address,
- bio,
- skills,
- GitHub URL,
- LinkedIn URL,
- `updated_at` timestamp.
- allows to change password

6.3 API Support

The profile feature is supported by:

- `GET /portfolio/me`
- `PUT /portfolio/me`

The implementation uses the authenticated member ID from the validated token rather than accepting an arbitrary member ID from the user. This ensures that users cannot modify another member's profile by manipulating request payloads.

6.4 UI Integration

The dashboard includes a `My Profile` button, which navigates to the profile page. The profile page loads member details and the editable profile fields, and allows the logged-in member to update their own information.

7 Audit Logging

7.1 Need for Persistent Auditability

The assignment required logging every modifying API action and being able to identify who changed what and when. Simple console or file logging is insufficient for this because it is not queryable at the data level and does not inherently capture before-and-after row states.

To address this, a database-backed audit table named `db_change_audit` was added.

7.2 Audit Table Design

The table stores the following fields:

- `table_name`
- `operation`
- `pk_value`
- `actor_member_id`
- `endpoint`
- `source`
- `before_json`
- `after_json`
- `changed_at`

This directly supports the audit requirement:

- **who**: `actor_member_id`
- **what**: `before_json` and `after_json`
- **when**: `changed_at`

7.3 Write Coverage

The audit mechanism was integrated with all important modifying application paths, including:

- ticket creation,
- ticket update,
- ticket delete,

- profile update.

The backend helper sets the required audit context variables before executing each write on the same database connection.

8 Tamper Detection via Triggers

8.1 Why Application Logs Alone Are Not Enough

A log written by the application can only record actions that pass through the application. It cannot, by itself, detect a direct SQL update executed from a database client. Because Module B explicitly asks for identification of unauthorized direct database modifications, a database-level mechanism is necessary.

8.2 Trigger Design

MySQL AFTER triggers were added for both `tickets` and `member_portfolio`:

- AFTER INSERT,
- AFTER UPDATE,
- AFTER DELETE.

For each change, the trigger inserts a row into `db_change_audit`. Trigger logic inspects the MySQL session variables provided by the application:

- if actor and endpoint metadata exist, the row is classified as `API`,
- otherwise it is classified as `DIRECT_DB`.

8.3 Before/After Payloads

The trigger payload behavior is:

- INSERT: `before_json = NULL`, `after_json = NEW row`
- UPDATE: `before_json = OLD row`, `after_json = NEW row`
- DELETE: `before_json = OLD row`, `after_json = NULL`

This makes the tamper-detection mechanism simultaneously useful for auditing.

8.4 Validation of Tamper Detection

Two different update paths were tested:

1. a manual SQL update on `tickets` generated an audit row with `source = DIRECT_DB`,
2. an authenticated profile update through `PUT /portfolio/me` generated an audit row with `source = API`, along with actor and endpoint information.

This confirms that the mechanism distinguishes between legitimate application activity and direct database changes.

9 Query Optimization and Indexing

9.1 Optimization Philosophy

The assignment required indexing based on actual usage patterns involving `WHERE`, `ORDER BY`, and `JOIN`. Therefore, the optimization work was driven by real application queries rather than generic database tuning.

9.2 High-Value Query Patterns

The main queries inspected were:

- authentication lookup by username,
- admin-role check using member-role joins,
- user ticket listing with `WHERE member_id = ?  ORDER BY created_at DESC`,
- admin ticket listing with `ORDER BY created_at DESC`.

Among these, the ticket listing queries were the most meaningful targets for indexing because they combine filtering and ordering in a way that is directly improved by appropriate BTREE indexes.

9.3 Final Index Set

The final useful indexes were:

- `idx_tickets_member_created_at` on `tickets(member_id, created_at)`
- `idx_tickets_created_at` on `tickets(created_at)`

9.4 Why These Indexes Were Chosen

The reasoning is as follows:

- `tickets(member_id, created_at)` supports the user-specific ticket query efficiently by combining the filter column and the sort column,
- `tickets(created_at)` supports global admin listing sorted by time,
- redundant indexes on uniquely constrained columns were avoided because they did not add meaningful value for the observed workload.

9.5 Redundant Index Removal

To make the optimization defensible, clearly redundant or low-value indexes were removed or avoided in the benchmark scripts. This prevents over-indexing and aligns the report with actual optimization logic rather than cosmetic index creation.

10 Benchmarking Methodology

10.1 Need for Reproducible Evidence

Simply claiming that indexing improves performance is not sufficient. Therefore, a reproducible benchmark pipeline was built in `performance_test.py`.

10.2 Pipeline Steps

The benchmark script performs the following workflow:

1. run `drop_indexes.sql` to establish the baseline state,
2. benchmark the selected application queries,
3. capture `EXPLAIN FORMAT=JSON` for the same queries,
4. run `create_indexes.sql` to enable the optimized state,
5. re-run the same benchmarks,
6. re-capture EXPLAIN plans,
7. save the artifacts under `performance/results/`.

10.3 Generated Artifacts

The final benchmark pipeline produces:

- `timings_before.csv`
- `timings_after.csv`
- `explain_before.json`
- `explain_after.json`
- `summary.md`

This makes the optimization process reproducible and inspectable.

11 Benchmark Results and Interpretation

The benchmark generated the following measured averages:

Table 1: Before vs After Indexing Benchmark Results

Query	Before Avg (ms)	After Avg (ms)	Improvement
LOGIN_QUERY	0.154893	0.097675	36.94%
ADMIN_CHECK_QUERY	0.069361	0.058842	15.17%
USER_TICKETS_QUERY	0.072165	0.065415	9.35%
ADMIN_TICKETS_QUERY	0.118839	0.110508	7.01%

11.1 Interpretation

The benchmark shows measurable improvement across all benchmarked queries. However, not all improvements should be interpreted equally.

The most meaningful optimization evidence comes from:

- `USER_TICKETS_QUERY`
- `ADMIN_TICKETS_QUERY`

These queries are directly aligned with the added indexes and therefore best demonstrate the value of workload-driven indexing.

The login and admin-check queries also show improvement, but the login query in particular is not the strongest demonstration of index value because MySQL simplifies parts of it aggressively during planning. Therefore, the ticket listing queries form the core optimization argument in this report.

12 EXPLAIN Query-Plan Analysis

12.1 Why EXPLAIN Was Used

The assignment explicitly required EXPLAIN or equivalent query-plan evidence. Therefore, `EXPLAIN FORMAT=JSON` was captured before and after indexing for each selected query.

12.2 Queries Analyzed

The EXPLAIN workflow was applied to:

- LOGIN_QUERY
- ADMIN_CHECK_QUERY
- USER_TICKETS_QUERY
- ADMIN_TICKETS_QUERY

12.3 Interpretation of Plans

The JSON plans confirm that the optimizer changes access behavior after indexing. The most important interpretation is not just that EXPLAIN files exist, but that the ticket queries - the most workload-relevant paths - are the clearest examples of useful indexing.

The EXPLAIN outputs were saved as:

- `performance/results/explain_before.json`
- `performance/results/explain_after.json`

These files serve as evidence accompanying the benchmark summary.

13 Testing and Validation

Following the style used in Assignment 1, the system was validated not only through successful flows but also through negative and adversarial cases.

13.1 Authentication and Route Protection Tests

The following access checks were performed:

- `/admin` without token returned 401 Unauthorized,
- `/admin` with a regular-user token returned 403 Forbidden,
- `/admin` with an admin token returned 200 OK,
- `/dashboard` without token returned 401 Unauthorized,

- `/dashboard` with a valid user token returned 200 OK.

  These tests confirm correct session validation and backend RBAC enforcement.

13.2 Audit and Tamper Tests

The following validation was performed:

- a manual SQL ticket update was executed in MySQL Workbench,
- the newest row in `db_change_audit` was verified to show `source = DIRECT_DB`,
- an authenticated profile update through the application was executed,
- the resulting audit row was verified to show `source = API`, correct `actor_member_id`, and endpoint metadata.

This confirms that the audit mechanism and tamper detection operate correctly in both normal and adversarial scenarios.

13.3 Benchmark Validation

The benchmark script was run successfully end-to-end and produced the expected result files. This confirmed:

- the baseline phase ran correctly,
- the indexed phase ran correctly,
- timing outputs were generated,
- EXPLAIN outputs were generated,
- the ticket indexes existed after the final optimized phase.

14 Tools and External Sources Used

The assignment instructions additionally asked for disclosure of tools and external sources used.

14.1 Tools Used

The following tools were used during implementation and testing:

- Python and Flask,
- MySQL and MySQL Workbench,
- VS Code,

- Git and GitHub,
- curl for HTTP route validation,
- LaTeX for report preparation.

14.2 External Assistance

The following external assistance was used:

- ChatGPT / Codex for implementation planning, debugging support, and report drafting assistance.

15 Repository README and Submission Notes

The instructions also require a README in the GitHub repository. The final repository should therefore clearly include:

- project structure,
- setup and dependency steps,
- database initialization steps,
- how to run the Flask app,
- how to run the benchmark pipeline.

The instructions further require a 5-minute video explaining the working of the system. Therefore, the final submission package should include or reference:

- the PDF report,
- the GitHub repository link,
- the video link,

16 Conclusion

Module B successfully extends the Assignment 1 database design into a working, secure, auditable, and performance-aware application.

The final implementation demonstrates:

- a functioning full-stack realization of the FixIIT schema,
- server-side authentication and role-based access control,
- database-backed write auditing and direct-database tamper detection,

- evidence-based query optimization using indexing, benchmarking, and EXPLAIN analysis.

Overall, the implementation satisfies the methodology, functionality, and approach requirements stated in the Assignment 2 instructions while remaining consistent with the domain model developed in Assignment 1.

17 Team Contributions

Table 2: Module-wise Team Contributions

Role	**Members**
Module A	Shiv Patel, Abhitej Singh Bhullar
Module B	Arin Mehta, Soham Shrivastava, Shivansh Shukla